

DOCUMENTATION PÉDAGOGIQUE

Systeme Anti-Triche JavaScript

Projet QCM Informatique — Version 2.0 corrigée

Ce document explique **ligne par ligne** le fichier `anti-triche.js` corrigé. Il décrit le rôle de chaque variable, de chaque fonction et de chaque événement JavaScript utilisé. Il présente également les bugs identifiés dans la version originale et les corrections apportées.

Objectif : vous permettre de comprendre et d'expliquer chaque ligne lors d'un examen ou d'une soutenance.

Table des matières

1.	Rapport d'audit – Bugs identifiés	3
2.	Architecture du fichier corrigé	5
3.	Section 1 – Configuration globale	6
4.	Section 2 – Plein écran	7
5.	Section 3 – Changement d'onglet / Blur	8
6.	Section 4 – Clic droit, Copier/Coller	10
7.	Section 5 – Raccourcis clavier interdits	11
8.	Section 6 – Gestion des avertissements (cœur du système)	13
9.	Section 7 – Actions de la modal	15
10.	Section 8 – Navigation entre questions	16
11.	Événements JavaScript expliqués	17
12.	Bonnes pratiques appliquées	19
13.	Instructions d'intégration	20

1. Rapport d'audit – Bugs identifiés

L'analyse du fichier `anti-triche.js` original a révélé **6 bugs** critiques ou majeurs, et 3 risques de contournement. Voici le détail complet.

#	Gravité	Description	Impact
B1	CRITIQUE	La soumission forcée est absente. Quand <code>MAX_AVERTISSEMENTS</code> est atteint, la modal s'affiche mais le QCM continue.	L'étudiant peut ignorer tous les avertissements.
B2	CRITIQUE	blur sur window déclenche un avertissement pendant que la modal est ouverte.	Double avertissement parasite à chaque infraction.
B3	MAJEUR	<code>MAX_AVERTISSEMENTS = 2</code> trop bas. Deux focus-outs accidentels = soumission.	Faux positifs fréquents (clic sur barre de tâches).
B4	MAJEUR	<code>copy/cut/paste/contextmenu</code> bloquent l'action sans avertir l'étudiant.	Contournement invisible, aucun warning comptabilisé.
B5	MAJEUR	<code>Ctrl+Shift+I</code> testé avec <code>e.key === "I"</code> (majuscule). Certains OS renvoient <code>"i"</code> minuscule.	La détection échoue sur Linux/Mac.
B6	MINEUR	<code>fullscreenchange</code> non préfixé. Safari utilise <code>webkitfullscreenchange</code> .	Sortie de plein écran non détectée sur Safari.
R1	RISQUE	Pas de garde anti-spam entre événements. <code>visibilitychange</code> + blur peuvent se cumuler.	Un seul onglet-switch compte double.
R2	RISQUE	<code>Ctrl+Shift+J</code> (Console Chrome), <code>F5</code> , <code>Ctrl+R</code> sont absents de la liste des raccourcis.	Ouverture de la console possible sans avertissement.
R3	RISQUE	<code>retourQCM()</code> ne remet pas <code>modalOuverte = false</code> .	Le garde blur reste désactivé après la première modal.

- **CRITIQUE** Fonctionnalité brisée ou contournable trivialement
- **MAJEUR** Comportement incorrect ou détection partielle
- **MINEUR** Problème de compatibilité navigateur
- **RISQUE** Vecteur de contournement non exploité actuellement

2. Architecture du fichier corrigé

Le fichier `anti-triche.js` version 2.0 est organisé en **8 sections** numérotées et documentées :

Section	Nom	Rôle
1	Configuration globale	Déclare les constantes et variables partagées
2	Plein écran	Active et surveille le mode plein écran
3	Changement d'onglet / Blur	Détecte la perte de focus
4	Clic droit, Copier/Coller	Bloque et signale les actions interdites
5	Raccourcis clavier	Intercepte les touches dangereuses
6	Gestion des avertissements	Cœur du système : compteur, modal, soumission
7	Actions de la modal	Boutons Reprendre / Abandonner
8	Navigation entre questions	Déplacement entre les 10 questions

3. Section 1 – Configuration globale

Ces lignes définissent les variables et constantes qui seront utilisées par tout le reste du fichier.

```
const MAX_AVERTISSEMENTS = 3;
```

const déclare une constante : sa valeur ne peut plus changer après initialisation. **MAX_AVERTISSEMENTS** fixe le seuil maximal d'infractions tolérées avant soumission forcée. La valeur **3** remplace le 2 originel qui était trop bas et provoquait des faux positifs.

■ **CORRECTION** : **MAX_AVERTISSEMENTS** passe de 2 à 3 (B3) pour réduire les faux positifs.

```
let avertissements = 0;
```

let déclare une variable modifiable (contrairement à **const**). **avertissements** est le compteur courant. Il commence à 0 et est incrémenté dans `enregistrerAvertissement()`.

```
let modalOuvrte = false;
```

Drapeau booléen (vrai/faux). Quand la modal s'ouvre, on le passe à `true`. L'écouteur `blur` vérifie ce drapeau avant de déclencher un avertissement, évitant ainsi le double-comptage.

■ **CORRECTION** : Correction du bug B2 : sans ce drapeau, chaque modal déclenchait un blur parasite.

```
let derniereDetection = 0;  
const DELAI_MIN_MS = 800;
```

derniereDetection mémorise l'horodatage (`Date.now()`) du dernier avertissement. **DELAJ_MIN_MS** est la durée minimale (800 ms) entre deux avertissements successifs. Si deux événements se déclenchent dans ce délai (ex. : `fullscreenchange` + `blur`), seul le premier est comptabilisé.

■ **CORRECTION** : Correction du risque R1 : garde anti-spam entre événements simultanés.

4. Section 2 – Plein écran

Le mode plein écran empêche visuellement l'étudiant de voir d'autres fenêtres. La sortie de ce mode déclenche un avertissement.

Fonction activerPleinEcran()

```
function activerPleinEcran() {
  const el = document.documentElement;
  if      (el.requestFullscreen)      el.requestFullscreen();
  else if (el.webkitRequestFullscreen) el.webkitRequestFullscreen();
  else if (el.mozRequestFullScreen)   el.mozRequestFullScreen();
}
```

document.documentElement retourne la balise <html>, c'est l'élément racine que l'on veut mettre en plein écran. Les trois conditions vérifient la disponibilité de l'API selon le navigateur :

- **requestFullscreen** — API standard (Chrome, Firefox, Edge)
- **webkitRequestFullscreen** — Préfixe pour Safari
- **mozRequestFullScreen** — Préfixe pour les anciennes versions de Firefox

Détection de sortie du plein écran

```
['fullscreenchange', 'webkitfullscreenchange', 'mozfullscreenchange'].forEach(evt => {
  document.addEventListener(evt, () => {
    const estPleinEcran = document.fullscreenElement
                        || document.webkitFullscreenElement
                        || document.mozFullScreenElement;
    if (!estPleinEcran) {
      enregistrerAvertissement('Vous avez quitté le mode plein écran.');
```

On écoute les trois variantes de l'événement pour couvrir tous les navigateurs. **document.fullscreenElement** retourne l'élément actuellement en plein écran, ou null si on en est sorti. L'opérateur || (OU logique) teste les trois propriétés : si elles sont toutes null, l'expression est fausse et on sait que le plein écran a été quitté.

■ **CORRECTION** : Correction du bug B6 : ajout des préfixes webkit et moz.

5. Section 3 – Changement d'onglet / Blur

visibilitychange

```
document.addEventListener('visibilitychange', () => {
  if (document.hidden) {
    enregistrerAvertissement('Vous avez quitté la fenêtre du QCM.');
```

L'événement **visibilitychange** se déclenche quand la visibilité de l'onglet change. **document.hidden** est un booléen : `true` si l'onglet n'est plus visible (changement d'onglet, minimisation, Alt+Tab), `false` sinon. C'est l'API officielle W3C pour détecter la perte de visibilité.

■ **INFO** : `visibilitychange` est déclenché par le navigateur, même si le code ne peut pas vraiment "bloquer" le changement d'onglet. Il sert uniquement à l'enregistrer.

blur sur window (perte de focus)

```
window.addEventListener('blur', () => {
  if (!modalOuvrte) {
    enregistrerAvertissement('Vous avez cliqué hors de la fenêtre du QCM.');
```

L'événement **blur** sur `window` se déclenche quand la fenêtre du navigateur perd le focus (clic dans une autre application, ouverture d'un menu OS, Alt+F4...). La différence avec `visibilitychange` : `blur` détecte la perte de focus sur le *bureau*, `visibilitychange` détecte le changement d'onglet *dans* le navigateur.

La condition `if (!modalOuvrte)` est le garde anti-rebond : quand la modal s'ouvre (et prend le focus), la fenêtre principale perd le focus, ce qui déclencherait normalement un événement `blur`. Sans ce garde, chaque infraction compterait double.

■ **CORRECTION** : Correction du bug B2 : le garde `modalOuvrte` empêche le double-comptage.

Comparaison `visibilitychange` vs `blur`

	<code>visibilitychange</code>	<code>blur (window)</code>
Quand ?	Changement d'onglet, minimisation	Clic dans une autre appli, Alt+Tab
Ciblé sur	<code>document</code>	<code>window</code>
<code>document.hidden</code> ?	Oui (utile)	Non (toujours <code>false</code>)
Bloquable ?	Non	Non
Cas typique	Étudiant ouvre un autre onglet	Étudiant bascule sur Word/Google

6. Section 4 – Clic droit, Copier/Coller

Ces écouteurs interceptent les actions susceptibles de permettre la copie de questions ou le collage de réponses trouvées en ligne.

```
document.addEventListener('contextmenu', e => {
    e.preventDefault();
    enregistrerAvertissement('Utilisation du clic droit interdite.');
```

```
});

document.addEventListener('copy', e => {
    e.preventDefault();
    enregistrerAvertissement('Copie de texte interdite.');
```

```
});

document.addEventListener('selectstart', e => e.preventDefault());
```

e.preventDefault() annule l'action par défaut du navigateur (affichage du menu contextuel, copie dans le presse-papiers...). L'objet **e** est l'événement passé automatiquement par le navigateur.

Événement	Déclenché quand	Action bloquée	Avertissement
contextmenu	Clic droit souris	Menu contextuel	Oui
copy	Ctrl+C ou copier	Copie presse-papiers	Oui
cut	Ctrl+X ou couper	Couper presse-papiers	Oui
paste	Ctrl+V ou coller	Coller depuis presse-papiers	Oui
selectstart	Début de sélection texte	Surlignage du texte	Non (silencieux)

■ **CORRECTION** : Correction du bug B4 : ajout des appels à enregistrerAvertissement() (absents en v1).

■ **INFO** : selectstart est silencieux car la sélection de texte peut être accidentelle (cliquer-glisser). On la bloque sans la pénaliser.

7. Section 5 – Raccourcis clavier interdits

Cette section intercepte les raccourcis qui permettraient d'ouvrir les outils développeur, de voir le code source ou de recharger la page.

Structure RACCOURCIS_INTERDITS

```
const RACCOURCIS_INTERDITS = [  
  { test: e => e.key === 'F12',  
    label: 'F12 (Outils développeur)' },  
  { test: e => e.ctrlKey && e.key.toLowerCase() === 'u',  
    label: 'Ctrl+U (Voir la source)' },  
  // ... autres raccourcis  
];
```

On utilise un **tableau d'objets** au lieu d'une longue suite de `if/else`. Chaque objet contient :

- **test** : une fonction flèche (`e => . . .`) qui reçoit l'événement et retourne `true` si ce raccourci est détecté
- **label** : une description lisible affichée dans le message d'avertissement

Ce design est **extensible** : pour ajouter un nouveau raccourci, il suffit d'ajouter un objet au tableau.

Propriétés utiles de l'objet événement (keydown)

Propriété	Type	Valeur exemple	Signification
<code>e.key</code>	string	"F12", "u", "l"	Touche pressée (sensible à la casse OS)
<code>e.key.toLowerCase()</code>	string	"f12", "u", "l"	Touche normalisée en minuscule (recommandé)
<code>e.ctrlKey</code>	boolean	true/false	Vrai si Ctrl est maintenu
<code>e.shiftKey</code>	boolean	true/false	Vrai si Shift est maintenu
<code>e.metaKey</code>	boolean	true/false	Vrai si Cmd (Mac) ou touche Win
<code>e.altKey</code>	boolean	true/false	Vrai si Alt est maintenu
<code>e.preventDefault()</code>	method	—	Annule l'action par défaut de la touche

Boucle de détection

```
document.addEventListener('keydown', e => {  
  for (const raccourci of RACCOURCIS_INTERDITS) {  
    if (raccourci.test(e)) {  
      e.preventDefault();  
      enregistrarAvertissement(`Raccourci interdit : ${raccourci.label}`);  
      return;  
    }  
  }  
});
```

On parcourt le tableau avec **for...of**. Pour chaque raccourci, on appelle `raccourci.test(e)`. Si la fonction retourne `true`, on bloque l'action et on enregistre l'avertissement. Le **return** final évite de comptabiliser plusieurs avertissements pour une seule frappe.

■ **CORRECTION** : Correction du bug B5 : `e.key.toLowerCase()` normalise la casse pour tous les OS.

■ **CORRECTION** : Correction du risque R2 : Ctrl+Shift+J, Ctrl+S, Ctrl+A, F5, Ctrl+R ajoutés.

8. Section 6 – Gestion des avertissements (cœur du système)

C'est la fonction centrale du système. Elle reçoit un message, l'enregistre, affiche la modal et, si nécessaire, soumet le formulaire.

```
function enregistrerAvertissement(message) {
    // 1. Garde anti-spam
    const maintenant = Date.now();
    if (maintenant - derniereDetection < DELAI_MIN_MS) return;
    derniereDetection = maintenant;

    // 2. Incrémentation
    avertissements++;

    // 3. Récupération des éléments DOM
    const modal = document.getElementById('avertissement-triche');
    if (!modal) return;
    const paragraphe = modal.querySelector('p');
    const btnReprendre = modal.querySelector('#btn-reprendre');

    // 4. Mise à jour du texte
    paragraphe.textContent =
        `${message} (Avertissement ${avertissements} / ${MAX_AVERTISSEMENTS})`;

    // 5. Label dynamique du bouton
    if (avertissements >= MAX_AVERTISSEMENTS) {
        if (btnReprendre) btnReprendre.textContent = 'Soumettre le QCM';
    } else {
        if (btnReprendre) btnReprendre.textContent = 'Reprendre le QCM';
    }

    // 6. Affichage de la modal
    modalOuvverte = true;
    modal.classList.remove('hidden');

    // 7. Soumission forcée
    if (avertissements >= MAX_AVERTISSEMENTS) {
        setTimeout(() => {
            const form = document.getElementById('qcm-form');
            if (form) form.submit();
            else window.location.href = '../index.php';
        }, 2000);
    }
}
```

Explication ligne par ligne

<code>Date.now()</code>	Retourne l'horodatage actuel en millisecondes depuis le 1er janv. 1970.
<code>avertissements++</code>	Incrémente le compteur de 1 (post-increment : lit puis ajoute 1).
<code>getElementById()</code>	Retourne l'élément HTML dont l'id correspond au paramètre.
<code>querySelector()</code>	Retourne le premier élément correspondant au sélecteur CSS donné.

<code>textContent</code>	Modifie le texte d'un élément HTML (plus sûr que <code>innerHTML</code> contre les XSS).
<code>`\${...}`</code>	Template literal : chaîne avec interpolation de variable.
<code>classList.remove()</code>	Retire une classe CSS d'un élément (ici "hidden" qui masque la modal).
<code>setTimeout(fn, 2000)</code>	Exécute la fonction <code>fn</code> après 2000 ms (2 secondes).
<code>form.submit()</code>	Soumet le formulaire par programme, comme un clic sur le bouton Submit.

■ **CORRECTION** : Correction du bug B1 : soumission forcée avec `setTimeout(2000)` ajoutée.

9. Section 7 – Actions de la modal

retourQCM()

```
function retourQCM() {  
    modalOuverte = false;  
    document.getElementById('avertissement-triche').classList.add('hidden');  
    activerPleinEcran();  
}
```

Cette fonction est appelée par le bouton "Reprendre" de la modal.

1. **modalOuverte = false** : réactive le garde anti-rebond blur.
2. **classList.add("hidden")** : masque la modal (applique display:none via CSS).
3. **activerPleinEcran()** : re-demande le plein écran, car la modal l'a peut-être interrompu.

■ **CORRECTION** : Correction du risque R3 : modalOuverte = false absent en v1, le garde restait désactivé.

annulerTentative()

```
function annulerTentative() {  
    if (confirm('Abandonner cette tentative ?...')) {  
        window.location.href = '../..../index.php';  
    }  
}
```

confirm() affiche une boîte de dialogue native du navigateur avec un message et deux boutons (OK/Annuler). Retourne true si l'utilisateur clique OK. **window.location.href** redirige vers l'URL indiquée, ici la page d'accueil.

10. Section 8 – Navigation entre questions

```
let questionActuelle = 0;
const nbQuestions = document.querySelectorAll('.question').length;

function changerQuestion(direction) {
  const questions = document.querySelectorAll('.question');
  questions[questionActuelle].classList.remove('active');

  questionActuelle += direction;
  questionActuelle = Math.max(0, Math.min(questionActuelle, nbQuestions - 1));

  questions[questionActuelle].classList.add('active');
  document.getElementById('question-actuelle').textContent = questionActuelle + 1;
  document.getElementById('btn-precedent').disabled = (questionActuelle === 0);
  document.getElementById('btn-suivant').classList.toggle('hidden',
    questionActuelle === nbQuestions - 1);
  document.getElementById('btn-valider').classList.toggle('hidden',
    questionActuelle !== nbQuestions - 1);
}
```

querySelectorAll('.question') retourne tous les éléments ayant la classe CSS "question" sous forme de NodeList.

.length retourne leur nombre (10 dans ce projet).

classList.remove('active') / **classList.add('active')** : bascule la visibilité via CSS (.question {display:none} / .question.active {display:block}).

Math.max(0, Math.min(...)) : "clamping" — empêche d'aller en dessous de 0 ou au-dessus de nbQuestions - 1.

classList.toggle('hidden', condition) : ajoute "hidden" si condition est vraie, la retire sinon. Permet d'afficher/masquer les boutons Suivant et Valider.

11. Événements JavaScript expliqués

Un **événement** est un signal émis par le navigateur quand quelque chose se passe (clic, frappe, chargement...). **addEventListener(type, fonction)** associe une fonction à exécuter quand cet événement survient.

Événement	Cible	Déclenché quand
DOMContentLoaded	document	Le HTML est entièrement analysé (DOM prêt, images non attendues)
visibilitychange	document	L'onglet change de visibilité (hidden/visible)
blur	window	La fenêtre du navigateur perd le focus système
fullscreenchange	document	Entrée ou sortie du mode plein écran
contextmenu	document	Clic droit ou touche Menu
copy / cut / paste	document	Action copier / couper / coller
selectstart	document	Début de sélection de texte par la souris
keydown	document	Touche du clavier enfoncée (avant la répétition)

Schéma du flux d'un avertissement

Voici la chaîne complète depuis la détection jusqu'à la sanction :

Étape	Action	Variable/Fonction concernée
1	Étudiant réalise une action interdite	(onglet, blur, raccourci...)
2	Événement JS déclenché	visibilitychange / blur / keydown...
3	Vérification du délai anti-spam	Date.now() - derniereDetection < 800ms ?
4	Incrémentation du compteur	avertissements++
5	Affichage de la modal	modal.classList.remove("hidden")
6	Vérification du seuil	avertissements >= MAX_AVERTISSEMENTS ?
7a (non)	Étudiant ferme la modal et reprend	retourQCM() → modalOuverte = false
7b (oui)	Soumission forcée après 2 secondes	setTimeout → form.submit()

12. Bonnes pratiques appliquées

const vs let

const pour les valeurs immuables (MAX_AVERTISSEMENTS), let pour les variables (avertissements). Cela exprime l'intention du développeur et prévient les bugs d'assignation.

Fonctions nommées et commentées

Chaque fonction a un commentaire JSDoc expliquant son rôle, ses paramètres et les corrections apportées.

Design pattern tableau d'objets

RACCOURCIS_INTERDITS utilise un tableau d'objets au lieu de if/else imbriqués. Plus lisible, plus extensible, plus testable.

Garde anti-spam (debounce simplifié)

Le délai DELAI_MIN_MS évite les doublons d'événements qui se déclenchent simultanément (visibilitychange + blur).

Vérification de l'existence du DOM

if (!modal) return; évite que le script plante si l'élément HTML n'existe pas.

textContent plutôt qu'innerHTML

textContent protège contre les attaques XSS (injection de HTML malveillant dans le message).

Aucune dépendance externe

Le fichier utilise uniquement les APIs natives du navigateur. Pas de jQuery, pas de framework. Zéro téléchargement supplémentaire.

Compatibilité multi-navigateurs

Tous les événements et APIs préfixés (webkit, moz) sont gérés.

Feedback utilisateur clair

Le message d'avertissement inclut le compteur dynamique (X/MAX) pour que l'étudiant comprenne sa situation.

Séparation des responsabilités

Chaque section du fichier a une responsabilité unique et clairement définie.

13. Instructions d'intégration

Voici les deux modifications à effectuer pour intégrer la version corrigée :

Fichier 1 : Remplacer anti-triche.js

```
Chemin : qcm_project/public/js/anti-triche.js  
Action : Remplacer entièrement par le fichier anti-triche.js fourni
```

Fichier 2 : Modifier la modal dans qcm.php

Dans `qcm_project/pages/user/qcm.php`, remplacer la `div id="avertissement-triche"` existante par :

```
■ Avertissement  
Vous avez quitte la fenetre du QCM.  
Reprendre le QCM  
Abandonner
```

La seule modification est l'ajout de `id="btn-reprendre"` sur le premier bouton. Cet identifiant est nécessaire pour que `enregistrerAvertissement()` puisse mettre à jour dynamiquement le libellé du bouton au dernier avertissement.

Aucune autre modification requise

Les fichiers `timer.js`, `style.css`, `resultats.php` et tous les autres fichiers PHP restent **inchangés**.

Checklist de validation

- 1. Ouvrir `qcm.php` dans un navigateur
- 2. Vérifier que le plein écran s'active automatiquement
- 3. Changer d'onglet → modal avec "Avertissement 1/3" doit apparaître
- 4. Reprendre et changer d'onglet deux fois → "Avertissement 3/3" + soumission auto après 2s
- 5. Tester F12, Ctrl+Shift+I → modal déclenchée
- 6. Clic droit → modal déclenchée
- 7. Ctrl+C sur du texte → modal déclenchée